

07 System of Linear Equations



7.3 Elementary Row Operations

Augmented Matrices and Gaussian Elimination: Solving Systems and Inverting Matrices by Hand

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills You Will Master

1

Master the three elementary row operations: row swapping, row scaling, row sum

3

Reduce the augmented matrix with elementary row operations to solve a linear system

5

Understand the pivot concept, and why partial pivoting (row swapping) matters

7

Implement Gaussian elimination in Python to solve a linear system

2

Understand how to build the augmented matrix $[A \mid b]$

4

Watch lines, matrix, augmented matrix, and linear combination change together during solving

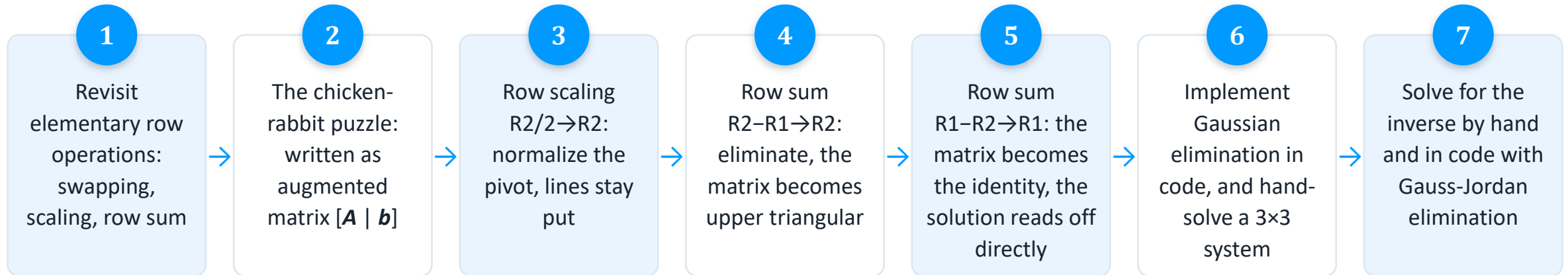
6

Solve for a matrix inverse by hand using elementary row operations (Gauss-Jordan elimination)

8

Implement elementary row operations in Python to compute a matrix inverse

From Augmented Matrices to Gaussian Elimination and Inverses



Elementary Row Operations

- ① Row swapping: exchange two chosen rows
- ② Row scaling: multiply every entry of one row by a nonzero scalar
- ③ Row sum: add a nonzero multiple of one row to another row

Fig 1 · The Original System, Linear Combination, Augmented Matrix

- 3 heads, 8 feet: $x_1 + x_2 = 3$, $2x_1 + 4x_2 = 8$ — two slanted lines, a linear combination of $\mathbf{a}_1$ and $\mathbf{a}_2$

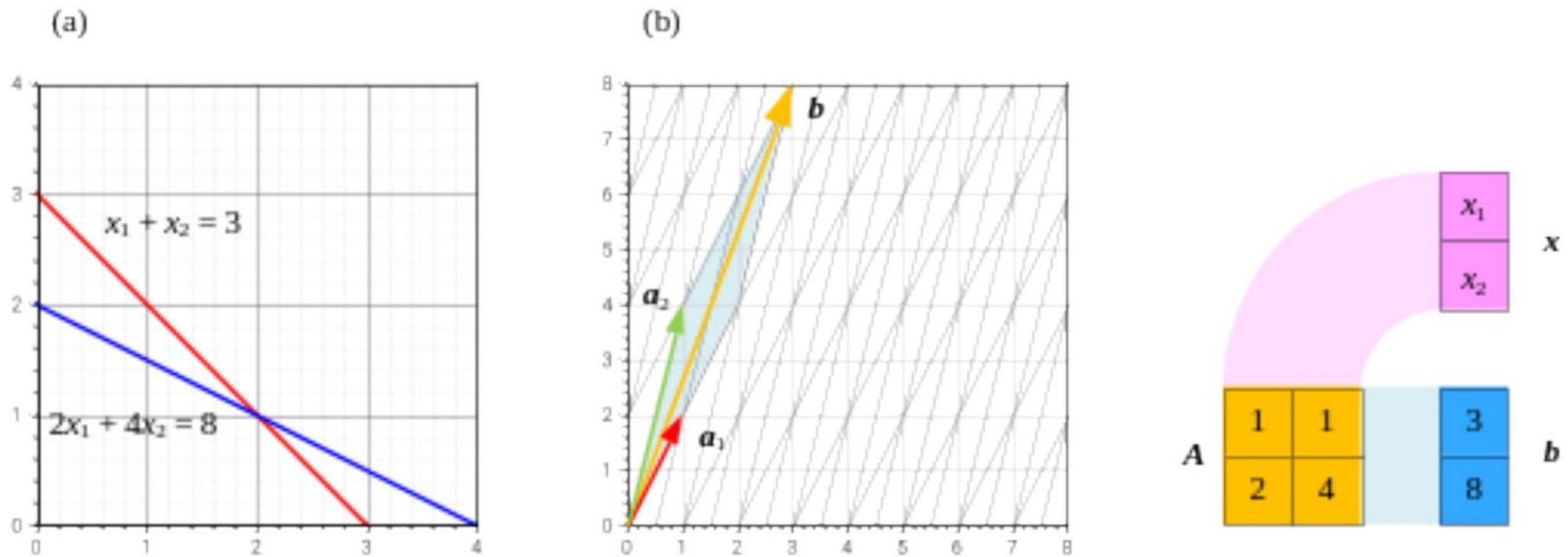


图 1. 原始方程组

$[A \mid b]$: Coefficient Matrix Joined with the Constants

$$[A \mid b] = [[1, 1 \mid 3], [2, 4 \mid 8]]$$

- The augmented matrix appends the constant vector b to the right of A — the dividing line is sometimes omitted
- Applying elementary row operations to the augmented matrix solves the linear system $Ax=b$

a_1, a_2 Are Not Collinear, $\det(A)=2$

$$x_1 a_1 + x_2 a_2 = b$$

- The angle between a_1, a_2 is under 90° but they are independent; the grid is a parallelogram and A has rank 2

$R_2/2 \rightarrow R_2$

Row Scaling: Normalizing the Pivot

$$R_2/2 \rightarrow R_2 \Rightarrow x_1 + x_2 = 3, x_1 + 2x_2 = 4$$

- Dividing row 2 by 2 and replacing it makes the pivot equal 1, which simplifies the steps that follow

Fig 2 · Row Scaling Leaves the Lines Unchanged

- Comparing Fig 1(a) and Fig 2(a): the two lines are identical — scaling just rescales both sides of the equation

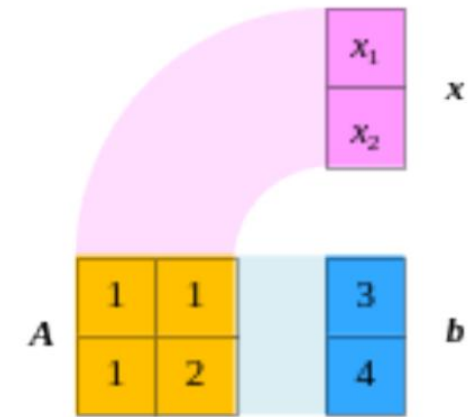
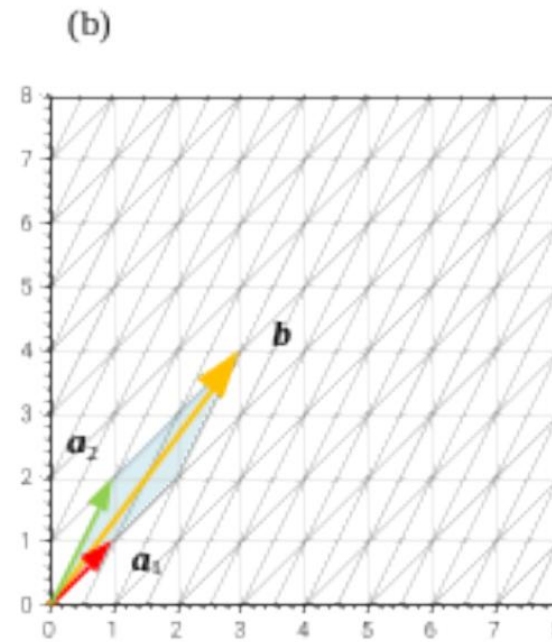
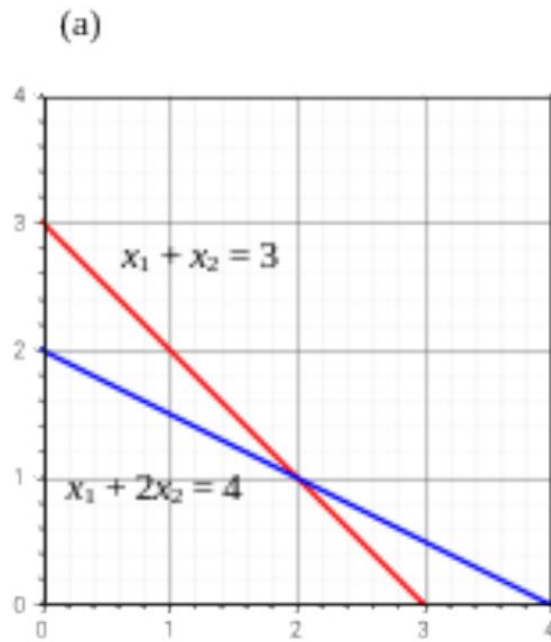


图 2. 行数乘, $R_2/2 \rightarrow R_2$

R2 - R1 → R2

Row Sum: Eliminating x_1 from Row 2

$$R_2 - R_1 \rightarrow R_2 \Rightarrow x_1 + x_2 = 3, x_2 = 1$$

- Row 2 minus row 1 replaces row 2; $x_2=1$ gives the solution directly, and $\mathbf{A}$ is now upper triangular

Fig 3 · The Grid Shears Along the Horizontal Axis

- $x_2=1$ is a horizontal line — the solution for x_2 has already appeared; the grid is now sheared, not parallelogram

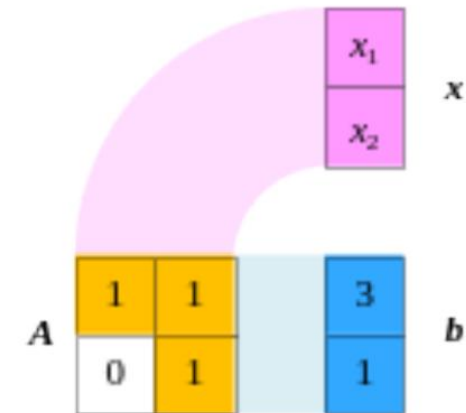
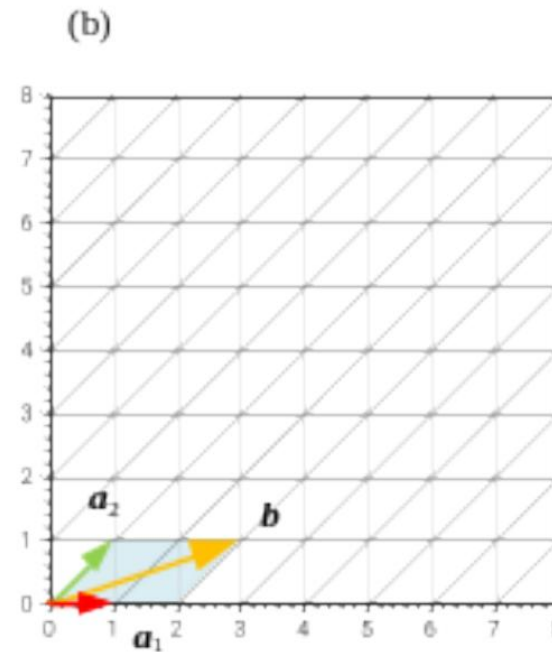
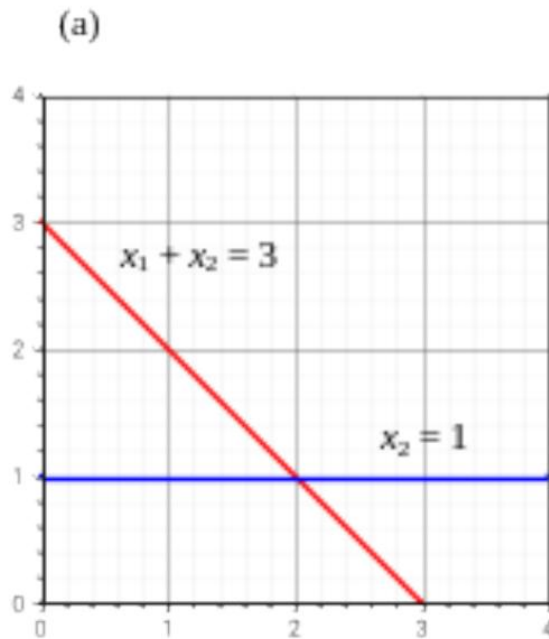


图 3. 行倍加, $R_2 - R_1 \rightarrow R_2$

$R_1 - R_2 \rightarrow R_1$

Row Sum: A Becomes the Identity I

$$R_1 - R_2 \rightarrow R_1 \Rightarrow x_1=2, x_2=1$$

- Row 1 minus row 2 replaces row 1; once $A=I$, $Ax=b$ simplifies directly to $x=b$

Fig 4 · The Grid Becomes a Unit Square, Solution Read Off

- Both lines are now vertical and horizontal; the two columns became the standard basis $[e_1, e_2]$

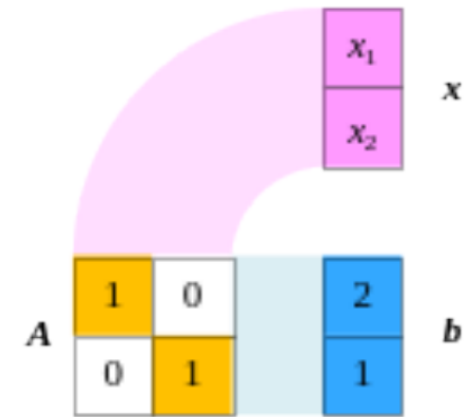
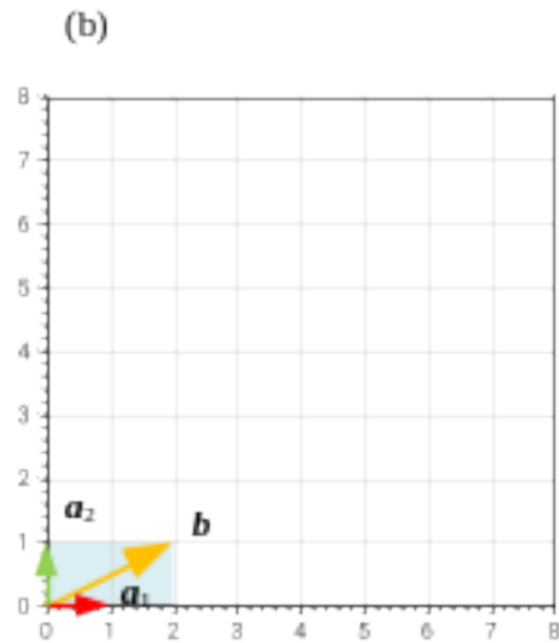
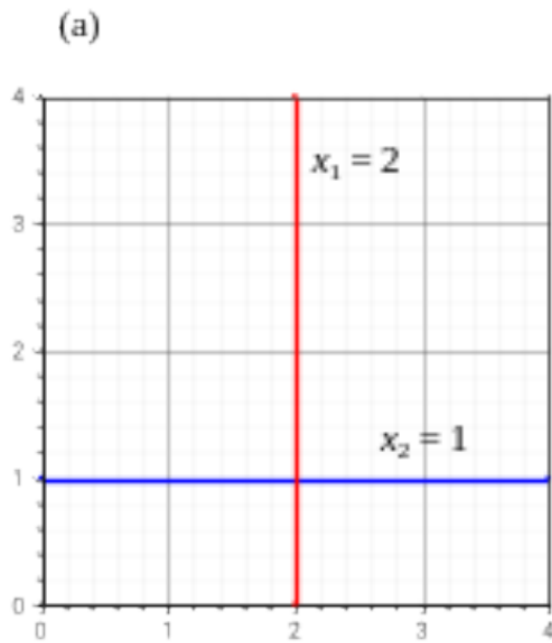


图 4. 行倍加, $R_1 - R_2 \rightarrow R_1$

SUMMARY

What Did Row Reduction Show Us?

Equations: two slanted lines gradually become horizontal and vertical, meeting at the solution point

Augmented matrix $[A \mid \mathbf{b}]$: the left reduces to I , the right column becomes the solution

Matrix A : from a general square matrix to upper triangular, and finally to the identity I

Linear combination: the grid shears and rescales step by step into a unit square

Solving a Linear System with Row Operations

gauss_elimination(A, b): Gaussian elimination with partial pivoting

- for i in range(n): loop over columns, reducing A to upper triangular one column at a time
- Row swap: pick the row with the largest absolute value in the current column as the pivot row
- Row scale: $Ab[i]=Ab[i]/Ab[i,i]$ normalizes the pivot to 1
- Row sum: $Ab[j]=Ab[j]-Ab[j,i]*Ab[i]$ eliminates the current column below the pivot
- Back substitution: starting from the last row, solve each component of x going upward

```
## 初始化
import numpy as np

## 自定义函数
def gauss_elimination(A, b):
    # 确保使用浮点数计算
    A = A.astype(float)
    b = b.astype(float)
    n = len(b)

    # 构造增广矩阵 [A|b]
    Ab = np.hstack([A, b.reshape(-1, 1)])

    # 原始矩阵A > 上三角
    for i in range(n):
        # 1) 行交换, 选择绝对值最大为主元
        # 主元: 主对角线元素
        max_row = np.argmax(np.abs(Ab[i:, i])) + i
        Ab[[i, max_row]] = Ab[[max_row, i]]
        # 交换行, 行 i <-> 行 max_row

        # 2) 行数乘, 归一化主元, 主元变为 1
        Ab[i] = Ab[i] / Ab[i, i]

        # 3) 行倍加, 消元, 使主元下面所有行的 i列变为 0
        for j in range(i+1, n):
            # Ab[j] -= Ab[j, i] * Ab[i]
            Ab[j] = Ab[j] - Ab[j, i] * Ab[i]

    # 回代求解, 上三角矩阵 > 单位矩阵
    x = np.zeros(n)
    for i in range(n-1, -1, -1):
        x[i] = Ab[i, -1] - np.dot(Ab[i, i+1:n], x[i+1:n])

    return x

## 定义矩阵 A 和向量 b
A = np.array([[1, 3, 6],
              [2, 7, 14],
              [0, 2, 5]])

b = np.array([25, 58, 19])

## 求解 Ax = b
x = gauss_elimination(A, b)
```

Swap → Scale → Row Sum

$$[[1,3,6|25],[2,7,14|58],[0,2,5|19]] \rightarrow [[1,3.5,7|29],[0,-0.5,-1|-4],[0,2,5|19]]$$

- Row 2 has the largest entry in column 1, so swap R_1, R_2 ; normalize with $R_1/2 \rightarrow R_1$; then $R_2 - R_1 \rightarrow R_2$ clears column 1

Swap → Scale → Row Sum

$$[[1, 3.5, 7 | 29], [0, -0.5, -1 | -4], [0, 2, 5 | 19]] \rightarrow [[1, 3.5, 7 | 29], [0, 1, 2.5 | 9.5], [0, 0, 0.25 | 0.75]]$$

- Row 3 has the largest entry below the column-2 pivot, so swap R_2, R_3 ; normalize $R_2/2 \rightarrow R_2$; then clear column 2

WORKED EXAMPLE · COLUMN 3 + BACK SUBSTITUTION

$$x_3=3, x_2=2, x_1=1$$

$$x_3=3 \Rightarrow x_2=9.5-2.5\times 3=2 \Rightarrow x_1=29-3.5\times 2-7\times 3=1$$

- **A** has only 3 columns, so column 3 needs only a row scale; then back substitution solves each variable in turn

Fig 5 · The Elementary Row Operation Pipeline

- A 's pivots (green) are normalized and eliminated column by column until the whole matrix turns green



图 5. 初等行变换的流程

Append I to A to Build an Augmented Matrix

$$[A \mid I] = [[1, 1 \mid 1, 0], [2, 4 \mid 0, 1]]$$

- Elementary row operations solve more than systems — the goal here is to turn the left side A into I , so the right side becomes A^{-1}

REPEATING THE SAME ROW OPERATIONS

$$[A \mid I] \rightarrow [I \mid A^{-1}]$$

$$[A \mid I] \rightarrow [[1,1|1,0],[0,1|-1,1/2]] \rightarrow [[1,0|2,-1/2],[0,1|-1,1/2]]$$

- Apply $R_2/2 \rightarrow R_2$, then $R_2 - R_1 \rightarrow R_2$, then $R_1 - R_2 \rightarrow R_1$ — the exact same three steps used to solve the system

VERIFICATION

$$\mathbf{A}\mathbf{A}^{-1}=\mathbf{I}, \mathbf{A}^{-1}\mathbf{A}=\mathbf{I}$$

$$\mathbf{A}^{-1} = \begin{bmatrix} 2 & -0.5 \\ -1 & 0.5 \end{bmatrix}$$

- Whether $\mathbf{A}$ multiplies $\mathbf{A}^{-1}$ or $\mathbf{A}^{-1}$ multiplies $\mathbf{A}$, the result is the identity $\mathbf{I}$ — the inverse checks out

Solving for a Matrix Inverse with Row Operations

inverse_matrix(A): Gauss-Jordan elimination for the inverse

- First check that $\mathbf{A}$ is square and $\det(\mathbf{A}) \neq 0$; otherwise return None
- $\mathbf{A_I} = \text{np.hstack}((\mathbf{A}, \mathbf{I}))$: build the augmented matrix $[\mathbf{A} \mid \mathbf{I}]$
- for i in $\text{range}(n)$: pick a pivot, normalize it, then eliminate every other row in that column
- Unlike solving a system: here every row above and below the pivot is eliminated, not just below
- $\mathbf{A_inv} = \mathbf{A_I}[:, n:]$: slicing out the right half of the augmented matrix gives $\mathbf{A}^{-1}$

```
## 初始化
import numpy as np

## 自定义函数
def inverse_matrix(A):

    # 确保 A 是 NumPy 数组并转换为浮点数类型
    A = np.array(A, dtype=float)

    # 获取矩阵的行数和列数
    n, m = A.shape

    # 判断是否为方阵
    if n != m:
        print("错误: 矩阵不是方阵, 无法求逆。")
        return None

    # 计算行列式, 判断是否可逆
    if np.linalg.det(A) == 0:
        print("错误: 矩阵的行列式为 0, 无法求逆。")
        return None

    # 创建单位矩阵 I
    I = np.eye(n)

    # 构造增广矩阵 [A | I]
    A_I = np.hstack((A, I))

    # 进行高斯-约当消元法
    for i in range(n):
        # 1) 行交换, 选取主元
        max_row = np.argmax(np.abs(A_I[i:, i])) + i
        A_I[[i, max_row]] = A_I[[max_row, i]]

        # 2) 行数乘, 归一化主元 (主对角线元素变为 1)
        A_I[i] = A_I[i] / A_I[i, i]

        # 3) 行加倍, 消元 (使主元列的其他元素变为 0)
        for j in range(n):
            if i != j:
                A_I[j] -= A_I[j, i] * A_I[i]

    # 提取逆矩阵
    A_inv = A_I[:, n:]

    return A_inv

## 定义矩阵 A
A = np.array([[1, 3, 6],
              [2, 7, 14],
              [0, 2, 5]])

## 计算逆矩阵
A_inv = inverse_matrix(A)
```

Row Operations: One Tool for Solving Systems and Inverting Matrices

$$[A \mid b] \rightarrow [I \mid x]; [A \mid I] \rightarrow [I \mid A^{-1}]$$

- Row swapping, scaling, and row sum — via partial-pivot Gaussian elimination — reduce A to I step by step
- The same operations, applied to $[A \mid b]$, solve a system; applied to $[A \mid I]$, they invert a matrix

Key Terms in This Section

Elementary Row Operations *Row swapping, row scaling, row sum*

Row Scaling *Multiplying one row by a nonzero scalar*

Augmented Matrix $[A \mid b]$ — *the coefficient matrix joined with the constants*

Partial Pivoting *Choosing the largest-magnitude entry in a column as the pivot to reduce error*

Gauss-Jordan Elimination *Reduce A directly to I , commonly used to find an inverse*

Row Swapping *Exchanging two rows of a matrix*

Row Sum *Adding a nonzero multiple of one row to another*

Pivot *The leftmost nonzero coefficient in a row during elimination*

Gaussian Elimination *Reduce A to upper triangular with row operations, then back-substitute*

Back Substitution *Solving for the unknowns row by row, starting from the last row*

Extended Reading

Further Reading · Elementary Matrices

LA_07_03_03.ipynb — every row operation corresponds to left-multiplying by an elementary matrix

Further Reading · LU Decomposition

LA_07_03_04.ipynb — Gaussian elimination is essentially an LU decomposition of A

Further Reading · `scipy.linalg.lu()`

LA_07_03_05.ipynb — using SciPy for LU and LDL decompositions

Open-Source Repository

All code and figures for the book: <https://github.com/Visualize-ML>

Q1 – Q8

Q1 Solve this system with row operations, plotting how the lines and linear combination change: $x_1+x_2=4$, $2x_1+4x_2=12$.

Q3 Use row operations to find the rank of Q2's matrix, and relate rank to invertibility.

Q5 What is LU decomposition, and how does it relate to Gaussian elimination without row swaps?

Q7 What is LDL decomposition, and what kind of matrix does it apply to?

Q2 Solve for the inverse of $\mathbf{A}=[[1,1],[1,2]]$ using row operations, and verify $\mathbf{A}\mathbf{A}^{-1}=\mathbf{I}$.

Q4 What is an elementary matrix? Which elementary matrix corresponds to each row operation?

Q6 Teach yourself to compute an LU decomposition with `scipy.linalg.lu()`.

Q8 Teach yourself to compute an LDL decomposition with `scipy.linalg.ldl()`.

Q9 – Q16

Q9 Write a function `row_swap(M, i, j)` that swaps rows i and j of matrix M .

Q11 Write a function `row_add(M, i, j, k)` that adds k times row j to row i .

Q13 Modify Code 1's `gauss_elimination` to also print the augmented matrix after every step.

Q15 Construct a case where partial pivoting fails (e.g. a column of all zeros) and test Code 1's function.

Q10 Write a function `row_scale(M, i, k)` that multiplies row i of M by a nonzero scalar k .

Q12 Combine `row_swap`, `row_scale`, `row_add` to hand-reduce this section's 3×3 example to upper triangular.

Q14 Solve this section's 3×3 example with `numpy.linalg.solve()` and compare to Code 1's hand-written result.

Q16 Use `matplotlib` to animate how the lines in Figs 1-4 change step by step under row operations.

Q17 – Q25

Q17 Write a function `is_upper_triangular(M)` that checks whether a matrix is upper triangular.

Q19 Modify Code 2's `inverse_matrix` to also print the augmented matrix after every step.

Q21 Construct a singular matrix (determinant 0) and test that Code 2's function correctly returns `None`.

Q23 Compare the computational cost of Gaussian elimination (with back substitution) versus Gauss-Jordan (straight to the identity).

Q25 Pick any invertible 4×4 matrix and find its inverse by hand (or in code) using row operations.

Q18 Use numpy to verify that an upper triangular matrix's determinant equals the product of its diagonal.

Q20 Solve this section's 2×2 example with `numpy.linalg.inv()` and compare to Code 2's hand-written result.

Q22 Write a function `gauss_jordan(A, b)` that solves a linear system directly via Gauss-Jordan, without back substitution.

Q24 Implement in Python the three elementary matrices for row swap, row scale, and row sum, and verify left-multiplication matches the row operation.

10 Prompts to Explore with AI

1 Why does Gaussian elimination pick the largest-magnitude entry as the pivot instead of always using the diagonal one?

2 Prove that every elementary row operation is equivalent to left-multiplying by a specific elementary matrix.

3 Explain the correspondence between the augmented matrix $[A \mid b]$ and the linear system $Ax=b$.

4 Use Python to hand-solve a 3×3 system with row operations, printing the augmented matrix at every step.

5 Explain the difference between Gaussian elimination and Gauss-Jordan elimination, and when each is preferred.

10 Prompts to Explore with AI

6

Explain why finding an inverse requires clearing entries both above and below the pivot, not just below.

7

Explain the relationship between LU decomposition and Gaussian elimination, and what L and U each represent.

8

Give a real-world example where solving a linear system or inverting a matrix matters in engineering or data science.

9

Use NumPy to benchmark a hand-written Gaussian elimination against `numpy.linalg.solve()` on large matrices.

10

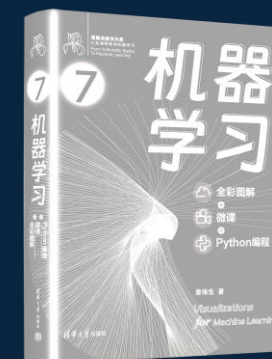
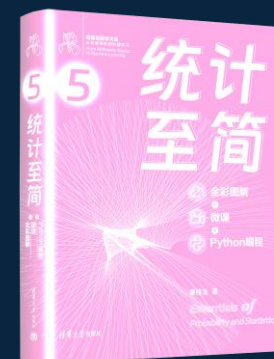
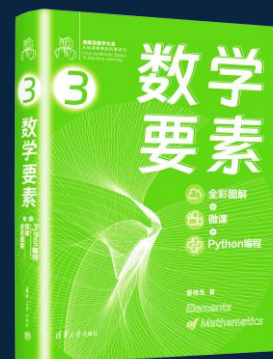
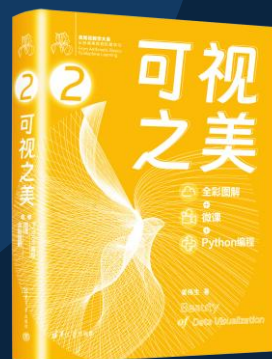
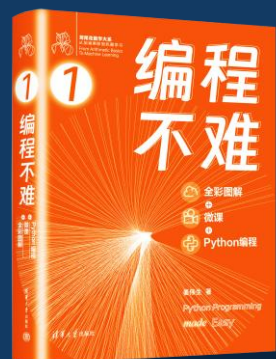
Design a small Python project: given any invertible $n \times n$ matrix, walk through the full row-reduction process to its inverse.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

